



**35<sup>th</sup>** Annual **INCOSE**  
international symposium  
hybrid event  
Ottawa, Canada  
July 26 - 31, 2025

# Enhancing Model-Based Systems Engineering with Large Language Models

Khushnood Adil Rafique  
Chair of Cyber-Physical-Systems,  
University of Kaiserslautern-Landau  
(RPTU),  
Kaiserslautern, Germany  
[khushnood.rafique@cs.uni-kl.de](mailto:khushnood.rafique@cs.uni-kl.de)

Sanan Shah  
Chair of Cyber-Physical-Systems,  
University of Kaiserslautern-Landau  
(RPTU),  
Kaiserslautern, Germany  
[shahs@rptu.de](mailto:shahs@rptu.de)

Šandor Dalecke  
Chair of Cyber-Physical-Systems,  
University of Kaiserslautern-Landau  
(RPTU),  
Kaiserslautern, Germany  
[sandor.dalecke@cs.uni-kl.de](mailto:sandor.dalecke@cs.uni-kl.de)

Christoph Grimm  
Chair of Cyber-Physical-Systems,  
University of Kaiserslautern-Landau  
(RPTU),  
Kaiserslautern, Germany  
[grimm@cs.uni-kl.de](mailto:grimm@cs.uni-kl.de)

Copyright © 2025 by Khushnood Adil Rafique. Permission granted to INCOSE to publish and use.

**Abstract.** Model-Based Systems Engineering (MBSE) is a cornerstone of modern systems engineering, enabling the design and management of increasingly complex systems. However, creating models with modeling languages such as System Modeling Language (SysML) V2 remains a highly manual and time-intensive process. As system complexity increases and interdisciplinary collaboration becomes critical, streamlining workflows and accelerating the modeling process are more important than ever. This paper investigates the use of two classes of Large Language Models (LLMs)— the general-purpose reasoning model, GPT-4, and the domain specific fine-tuned model, CodeT5, to assist in generating SysML V2 models from natural language descriptions. Our findings reveal that while LLMs can generate initial "skeleton" models, relying solely on AI for entire model generation is not advisable due to critical errors and inconsistencies in the generated outputs. Instead, LLMs serve as effective tools for accelerating the modeling process by producing foundational prototypes that can be refined and improved through human intervention. With fine-tuning techniques for CodeT5 and prompt engineering for GPT-4, we evaluate their performance across key dimensions of syntax accuracy, semantic alignment, and structural correctness. Using a diverse set of metrics and statistical analyses, we identify the trade-offs between task-specific optimization (CodeT5) and broader contextual reasoning (GPT-4). The potential of LLMs as accelerators in MBSE workflows cannot be ignored with the “early” modeling capabilities they present. This work contributes to the growing intersection of AI and MBSE, offering



**35<sup>th</sup>** Annual **INCOSE**  
international symposium  
hybrid event  
Ottawa, Canada  
July 26 - 31, 2025

practical insights into the selection and application of LLMs in systems engineering and paving the way for future advancements in the field.

**Keywords.** SysML V2, Systems Engineering, Natural Language Processing, LLMs, Automatic Code Generation, CodeT5, GPT-4, ChatGPT, Artificial Intelligence

## Introduction

The rise of Artificial Intelligence (AI) in engineering disciplines has introduced transformative opportunities to automate time-intensive processes, enhance productivity, and tackle the growing complexity of modern systems. Model-Based Systems Engineering (MBSE), a foundational approach in systems engineering, depends on tools like Systems Modeling Language (SysML) V2 (Moberley, Rosenberg, & Weilkeins, 2024). SysML V2 provides a framework for creating structured representations of system requirements, behavior, and architecture, ensuring consistency and collaboration across diverse engineering teams. With its capability to model multiple facets of systems, SysML V2 is an indispensable tool in addressing the challenges of contemporary systems design. Despite its many advantages, generating SysML V2 models remains, from start to finish, a predominantly manual and time-intensive process, particularly for large and intricate systems. Automating the initial phases of *prototyping* or *skeleton model* generation using AI-powered tools offers significant potential for improving efficiency, reducing time, and enhancing scalability. In this paper, the terms *model skeleton* and *model prototype* are used interchangeably to refer to an initial, lightweight SysML V2 representation derived from natural language descriptions, typically containing foundational structural elements without full behavioral or parametric detail. Natural Language Processing (NLP) offers a compelling avenue for achieving this automation, enabling engineers to articulate system concepts in natural language that can subsequently be translated into structured SysML V2 syntax. Certain classes of Large Language Models (LLMs) such as OpenAI's GPT-4 (OpenAI, 2023) and Google's CodeT5 (Wang, Y., Liu, H., Jiang, Z., Neubig, G., & Rajani, N., 2021) have demonstrated remarkable capabilities in bridging the gap between natural language descriptions and formal modeling languages, to a large extent.

## SysML V2 & MBSE

SysML V2 is based on the Kernel Modeling Language (KerML) metamodel proposed to the object management group (OMG) (Object Management Group, 2024). SysML v2 extends KerML to provide a general-purpose modeling language by providing means to represent the requirements, structure and behavior of a system. In addition, SysML V2 allows for modeling of specification and verification cases. One key aspect of SysML V2 is the support of textual model representation and the graphical representation known from SysML diagrams in the past. This textual representation is similar to programming languages in structure and syntax, e.g., using curly brackets for encapsulation and following distinct statement structures. This textual representation is well suited for the use of AI since it's easy



to verify and validate the output and can easily be used for training purposes whereas the graphical representation introduced additional overhead.

In MBSE, models are the central artefacts created and communicated between stakeholders. These models represent the intended system and function as the single source of truth to verify and validate against. MBSE offers a number of advantages compared to the traditional document-based systems engineering, namely:

- **Early Analysis and Simulation:** Errors are increasingly costlier to fix the later they are found. Early creation and analysis of models allow for early error detection, making it significantly easier to fix.
- **Traceability and Consistency:** MBSE models include design elements, requirements, constraints, dependencies, possible behavior, use cases and variables which are all connected. This facilitates consistency checking and allows for the traceability of changes.
- **Automation and Integration:** The use of models allows for automation of various tasks, including updates or addition of requirements, generation of test cases or design synthesis.
- **Life-cycle Management:** MBSE models don't lose their application after a certain development stage. A model can be continuously updated throughout development but also deployment, keeping them relevant and making it possible to adapt to new challenges and requirements long after a systems deployment.

While LLMs have been extensively studied for general-purpose text generation and programming-related tasks, their application to MBSE, specifically for generating modeling languages, remains an underexplored area of research. This paper seeks to address this gap by presenting the first systematic comparison of two distinct classes of LLMs: the large-scale, general-purpose GPT-4 and the smaller, domain-specific, and fine-tunable CodeT5 at prototyping SysML V2 models. Through a series of experiments, we assess these models on their ability to generate accurate and semantically meaningful SysML V2 model prototypes from natural language descriptions. Our experiments are built on a custom dataset and evaluate performance across three critical dimensions: syntax accuracy, semantic alignment, and structural correctness. This study contributes to the field of AI-driven MBSE by offering practical insights into the selection and limitations of LLMs for systems engineering workflows. Our findings emphasize the trade-offs between relatively smaller, resource-efficient models like CodeT5 and larger, more versatile models like GPT-4, highlighting their respective strengths and weaknesses in specific scenarios. By demonstrating how these models can be integrated into MBSE practices, we not only advance the state of the art in automation but also lay the groundwork for future exploration of AI-enabled tools in systems engineering.



## Related Work

AI and NLP in particular have shown great potential in automating MBSE. Johns et al. developed an NLP-based framework to convert natural language policy documents into SysML models, achieving a precision of  $\sim 0.86$  and recall of  $\sim 0.66$  in identifying roles and responsibilities, reducing manual effort (Johns, Brian; Carroll, Kristina; Medina, Casey; Lewark, Rae; Walliser, James, 2024). Similarly, Riesener et al. proposed an NLP-driven approach to transform unstructured requirements into structured formats, enhancing interdisciplinary collaboration and early validation (Riesener, M.; Dölle, C.; Becker, A.; Gorbacheva, S.; Rebentisch, E.; Schuh, G., 2021). Zevio introduced a tool that uses deep learning to extract semantic triples from requirements, generating machine-readable knowledge graphs to integrate into MBSE workflows (Zevio, Stella, 2024). Akundi et al. analyzed MBSE research trends using text mining, identifying SysML as the most widely used modeling language and uncovering key application areas such as cyber-physical systems (Akundi, Aditya; Euresi, Daniel; Luna, Sergio; Ankobiah, Wilma; Lopes, Amit; Edinbarough, Immanuel, 2022). Chami et al. demonstrated a text-to-model framework for generating SysML elements from semi-structured input text, highlighting AI's role in improving MBSE adoption (Chami, Mohammad; Zoghbi, Christophe; Bruel, Jean-Michel, 2019). While these studies highlight the potential of AI and NLP in automating MBSE processes, they largely focus on specific tasks such as requirements extraction or the creation of semantic triples. In contrast, our work uniquely addresses the generation of SysML V2 model skeletons directly from natural language descriptions. By comparing two distinct classes of LLMs—our study fills a critical gap in understanding how AI can be leveraged to provide a head start into complex modeling tasks at scale. Our approach not only advances the state of AI-driven MBSE but also offers practical insights into selecting the right LLM class for specific applications, balancing resource constraints with performance needs.

## Methodology

The study emphasizes the comparison with two distinct approaches: (1) adapting CodeT5, pre-trained for model skeleton generation, through transfer learning and (2) harnessing GPT-4, with engineered prompts using in-context learning. Figure 1 provides an overview of the pipelines used for both models. The pipeline for CodeT5 includes data preprocessing, model fine-tuning, and automated evaluation, while GPT-4's workflow involves prompt design, API interaction, manual review, and comparative analysis. These complementary methodologies enable us to explore both domain-specific adaptation and zero-shot generalization in generating SysML V2 model skeletons. This approach not only highlights the adaptability of transformer-based models to novel domains but also evaluates the potential of GPT-4 as a general-purpose generator for structured modeling languages.

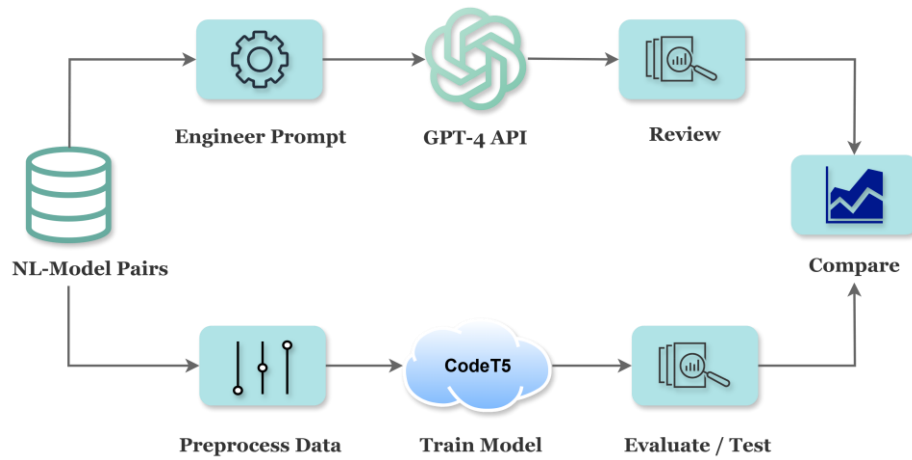


Figure 1. Pipeline of the Natural Language Sentences to SysML V2 Skeleton Model Generation.

## ***Training and Evaluating CodeT5***

Our initial experiments and tests involved fine-tuning CodeT5, a pre-trained transformer-based model, for the automatic generation of SysML V2 model skeletons from natural language descriptions. CodeT5, part of the encoder-decoder LLM family, is pre-trained on the CodeSearchNet dataset (Husain, H., Codio, C., Li, D., Napoles, C., Yang, S., Yasunaga, M., & Riedel, S., 2020), which includes code from nine popular programming languages. Its architecture is optimized for code understanding and generation tasks, making it a good candidate to represent domain specific, relatively smaller LLMs.

## ***Model Selection***

GPT-4 and CodeT5 represent two distinct classes of LLMs, each with unique design objectives and capabilities. GPT-4, has a massive parameter count, and is designed for a wide variety of tasks ranging from natural language understanding to creative writing and even code generation. In contrast, CodeT5, part of the CodeT5 family, is a smaller, task-specific model, fine-tuned for programming-related tasks. Together they embody the archetypes of their respective LLM classes. GPT-4 represents general-purpose LLMs that leverage their vast scale for contextual understanding and flexibility, while CodeT5 represents specialized, task-oriented LLMs optimized for resource efficiency and domain-specific tasks. This dichotomy allowed the study to evaluate both ends of the LLM spectrum, providing insights into the trade-offs between generality and specialization in the context of SysML V2 model skeleton generation. Their complementary nature made them good candidates for examining how LLMs can be leveraged in MBSE. By assessing their strengths and limitations, this study highlights when and why



an engineering team might choose one class of model over the other, depending on available computational resources, data, and the specific MBSE requirements. We compare the two models in more detail in the *Discussion* section of the paper.

## ***Dataset and Fine-Tuning Configuration of CodeT5***

For our study, a custom dataset of 2,000 samples was curated, where each sample consisted of a natural language description paired with the corresponding SysML V2 model. The dataset was limited because there was not an abundance of SysML V2 examples on the internet at the time of test preparation. Therefore, we hand crafted the majority of training samples while also making use of the SysML V2 Github repository. The primary objective was to train a mapping function  $f_\theta$ , parameterized by  $\theta$ , that translates natural language inputs  $x_i$  into their SysML V2 outputs  $y_i$ :

$$f_\theta(x_i) \approx y_i, \quad \forall (x_i, y_i) \in \mathcal{D}$$

The dataset, ( $\mathcal{D} = \{(x_i, y_i) \mid i = 1, 2, \dots, N\}$ ), was divided into training 80%, validation 10%, and testing 10% subsets. This partitioning ensured fair model evaluation, given our limited number of training samples and available computational resources. The fine-tuning process was conducted using the Hugging Face Transformers library (Wolf, 2020). Key hyperparameters included: Learning rate of ( $5 \times 10^{-5}$ ), batch size of 16, and the use of the *AdamW* Optimizer. The fine-tuning was performed on an NVIDIA A100 GPU.

Each code snippet in our dataset represented a simplified system typically described using a single SysML V2 package containing core structural elements such as Blocks, Parts, Associations, and Binding Connectors. The systems modeled ranged from basic cyber-physical systems (e.g., automotive sub-systems, HVAC control, robotic arms) to abstract representations like library or ticketing systems, which allowed for a diverse set of semantic and structural features to be captured. On average, each LLM-generated model contained 8–12 elements, including at least 2–3 blocks, interlinked by associations or part decompositions. These were designed to mirror the structure and complexity of models present in the SysML V2 training repository used in our study. While the repository includes approximately 20–25 reusable training models, we manually extended these with handcrafted variants to produce the sample pairs. The models generated during evaluation were generally single-package SysML V2 snippets, with 1–2 internal diagrams implicitly representing structural views. While we did not explicitly evaluate graphical diagrams, the textual SysML V2 representations encoded sufficient structure for modeling tools to render typical Block Definition and Internal Block Diagrams automatically. This ensures that the learning task remains both tractable and meaningful.





## ***Prompting and Reviewing GPT-4***

To adapt GPT-4 for this task, we designed a prompt that utilized in-context learning. The prompt was designed to include three templates from our training dataset, each consisting of a natural language description paired with its corresponding SysML V2 model. These examples were included to help GPT-4 better understand the task requirements and adapt its outputs accordingly. We experimented with *more than three* SysML V2 examples to provide more context for GPT-4, but did not record a significant improvement in the results. On the other hand, dropping below three examples, produced inferior results. Figure 2 represents the structure of the GPT-4. The prompt structure provided GPT-4 with sufficient context to generate accurate and reliable SysML V2 model skeletons. We also made sure to prompt GPT-4 to generate SysML V2 “models” as opposed to “model skeletons” as this produced the best results in our experiments. Using the term “model skeleton” or “model prototype” confused GPT-4 and it only produced basic structure of SysML V2 syntax that needed more manual revisions. Unlike CodeT5, GPT-4 does not rely on fine-tuning with a large dataset but instead leverages its pre-trained knowledge base and examples to generalize. Figure 3 gives an example of what one of the natural language descriptions along with its SysML V2 model, looked it during the prompting of GPT-4.

| <b>GPT-4 Prompt Structure</b>                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Instruction Prompt:</b> Based on the examples of SysML V2 models, and their respective natural language descriptions, shown below, analyze them the best you can, and with the knowledge you acquire, create a SysML V2 Model from the 'New NL description' |
| <b>Example 1:</b><br>Description. [NL Example 1]<br>SysML V2 Model: [SysML V2 Model Example 1]                                                                                                                                                                 |
| <b>Example 2:</b><br>Description. [NL Example 2]<br>SysML V2 Model: [SysML V2 Model Example 2]                                                                                                                                                                 |
| <b>Example 3:</b><br>Description. [NL Example 3]<br>SysML V2 Model: [SysML V2 Model Example 3]<br>'New NL Description': [NL Test Sample]                                                                                                                       |

Figure 2. GPT-4 Prompt for SysML V2 Model Prototype Generation.



**Description.** "The *State Actions* package models the operational behavior of a vehicle using a state machine driven by external signals. It defines three signal attributes—VehicleStartSignal, VehicleOnSignal, and VehicleOffSignal—alongside a part definition for a Vehicle. The core behavior is captured in the VehicleStates state machine, which operates on an input vehicle instance. Upon entry, the state machine transitions to the off state. When the VehicleStartSignal is received, the system moves from off to starting, and upon receiving the VehicleOnSignal, it transitions to the on state. In the on state, a performSelfTest action is executed on entry using the vehicle instance, followed by a continuous providePower action during the state, and an applyParkingBrake action upon exit. At any point, receiving the VehicleOffSignal causes the system to return to the off state. This structure ensures that the vehicle transitions through defined operational phases while responding to external control signals."

**SysML V2 Model:**

```
package 'State Actions' {
  attribute def VehicleStartSignal;
  attribute def VehicleOnSignal;
  attribute def VehicleOffSignal;

  part def Vehicle;

  action performSelfTest { in vehicle : Vehicle; }

  state def VehicleStates { in operatingVehicle : Vehicle; }

  state vehicleStates : VehicleStates {
    in operatingVehicle : Vehicle;

    entry; then off;

    state off;
    accept VehicleStartSignal
      then starting;

    state starting;
    accept VehicleOnSignal
      then on;

    state on {
      entry performSelfTest { in vehicle = operatingVehicle; }
      do action providePower { /... / }
      exit action applyParkingBrake { /... / }
    }

    accept VehicleOffSignal
      then off;
  }
}
```

Figure 3. One of the NL-Model pairs used in the GPT-4 prompt.





## Results

This section discusses the results and experiments conducted for an in-depth evaluation of two classes of LLMs—for generating SysML V2 model prototypes from natural language descriptions. By using diverse metrics across syntax accuracy, semantic alignment, and structural correctness, the experiments offer an overview of the models' capabilities. This analysis highlights their strengths and limitations and contextualizes their applicability within systems engineering workflows. We must also remember that we were relatively much more limited on the fine-tuning resources of CodeT5, compared to the training environment of GPT-4. Yet the final output of CodeT5 is surprisingly acceptable and also better than GPT-4 in many instances. Therefore, we again lay emphasis on the choice of the model class based on the use-case and the desired outcome.

### *Syntax Accuracy*

Syntax accuracy is crucial in evaluating automatic code generation, as syntax errors can lead to invalid models or downstream failures in MBSE workflows. To measure syntax accuracy, we use the Levenshtein Distance (Levenshtein, 1966), a metric that measures how many single-character edits (insertions, deletions, or substitutions) are needed to transform the generated code into the reference version. A lower Levenshtein Distance indicates that the generated snippet is syntactically closer to the expected output, making it a useful proxy for evaluating how faithfully the model reproduces valid code structures.

**Analysis of Levenshtein Distance Results.** Figure 4 compares the Levenshtein distances for SysML V2 model skeletons generated (represented as “snippets”) by GPT-4 and CodeT5. From the comparison, it is evident that GPT-4 achieves lower Levenshtein distances compared to CodeT5 for most snippets, indicating that GPT-4's outputs are usually closer to the reference models. However, exceptions (e.g., Snippet 5 and 9, which have more precise natural language descriptions) show CodeT5 outperforming GPT-4, suggesting task-specific advantages for the smaller and more task-optimized natural language-SysML V2 model pairs. CodeT5 exhibits greater variability in performance, due to its specialization for code-related tasks, while GPT-4 demonstrates stable performance across all examples, leveraging its general-purpose and larger model architecture. The analysis highlights the importance of selecting model classes based on natural language descriptions and their brevity. While GPT-4 generally provides better alignment with reference code with larger SysML V2 models, CodeT5 shows promise for specific scenarios, where the natural language descriptions are more precise.

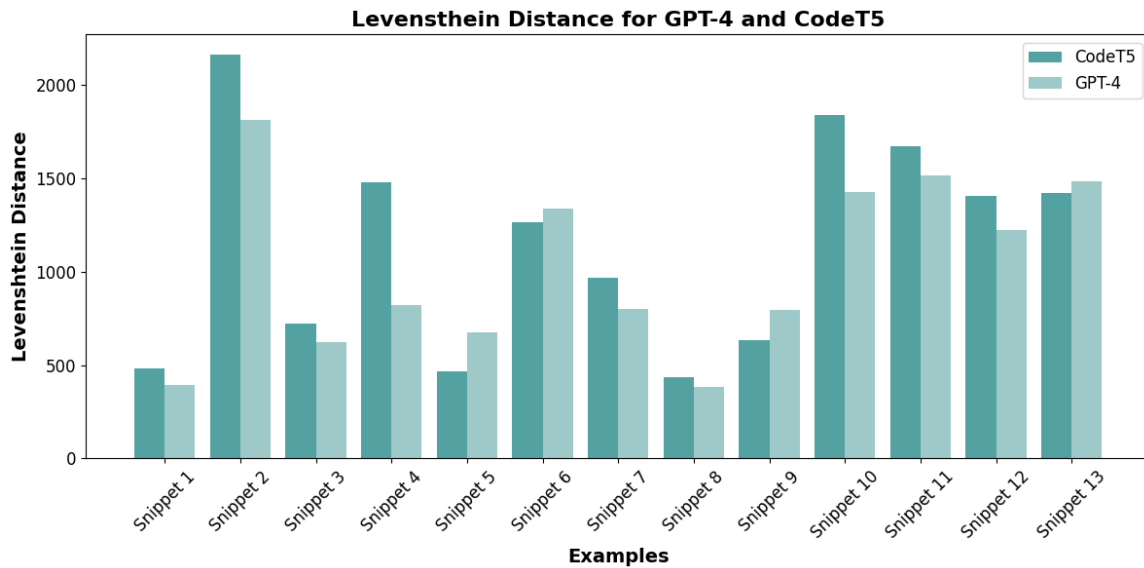


Figure 4. Comparison of Levenshtein Distances for GPT-4 and CodeT5 across Different Snippets.

## Semantic Alignment

Semantic alignment evaluates the extent to which generated models preserve the meaning and intent expressed in the natural language descriptions. In the context of MBSE, this is essential for ensuring that the functional and behavioral specifications are accurately represented in the system model. While syntactic correctness guarantees adherence to structural rules, semantic alignment assesses whether the generated code reflects the intended requirements. To quantify semantic alignment, we use Cosine Similarity (Manning, Christopher D.; Raghavan, Prabhakar; Schütze, Hinrich, 2008) and Jaccard Similarity (Jaccard, 1901), both of which are established measures in natural language processing and information retrieval. Cosine Similarity assesses the directional similarity between vector representations of textual inputs, capturing nuanced semantic relationships beyond exact word matching. Jaccard Similarity measures the overlap between sets of tokens, offering a complementary view based on shared lexical content. Together, these metrics provide an interpretable and scalable means of comparing the semantic content of generated and reference models, thus enabling a more comprehensive evaluation of alignment with the input specifications.

**Cosine Similarity Analysis.** Figure 5 compares the semantic similarity of snippets generated by CodeT5 and GPT-4 with respect to the reference code, measured by Cosine Similarity. The analysis reveals that GPT-4 achieves higher semantic similarity scores across most snippets, which signifies decent



alignment. In contrast, CodeT5 demonstrates acceptable performance with scores ranging from 0.75 to 0.87, albeit slightly lower on average compared to GPT-4. Despite these differences, CodeT5 still delivers commendable results, particularly in scenarios with clear and concise reference patterns. We must keep in mind that the goal is to generate first prototypes (or skeletons) of the SysML V2 models, which can be revised and corrected by human intervention.

**Jaccard Similarity Analysis.** The Jaccard similarity comparison, as illustrated in Figure 6 evaluates the extent of overlap between the generated model and the reference model based on the unique tokens in each. The results reveal an interesting dynamic between the two LLM models. CodeT5 demonstrates consistent performance across most snippets, showing its capability to replicate essential elements of the reference model. GPT-4, while achieving higher similarity in certain cases, exhibits greater variability, indicating a tendency to introduce additional or alternative syntax that might deviate from the reference structure.

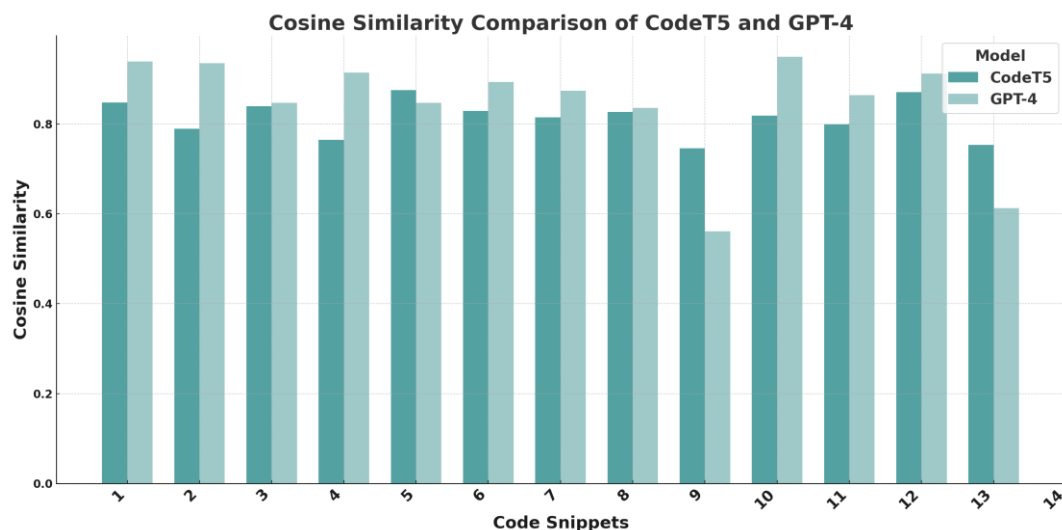


Figure 5. Comparison of Cosine Similarity Scores between CodeT5 and GPT-4 for Generated Snippets.

There is a trade-off between precision and creativity in model outputs. CodeT5's steadier performance shows its specialization in code-related tasks, whereas GPT-4's variance reflects its broader, more generalized modeling capabilities. These results emphasize that depending on the context—whether strict adherence to the reference syntax or flexibility in code generation is desired—the choice of a model class can have a decent impact. The Jaccard similarity findings align with the cosine similarity results, reinforcing the idea that both model classes have strengths suited to different applications. The insight informs the potential integration of AI in systems engineering, where balancing structural correctness and creative problem-solving is essential.

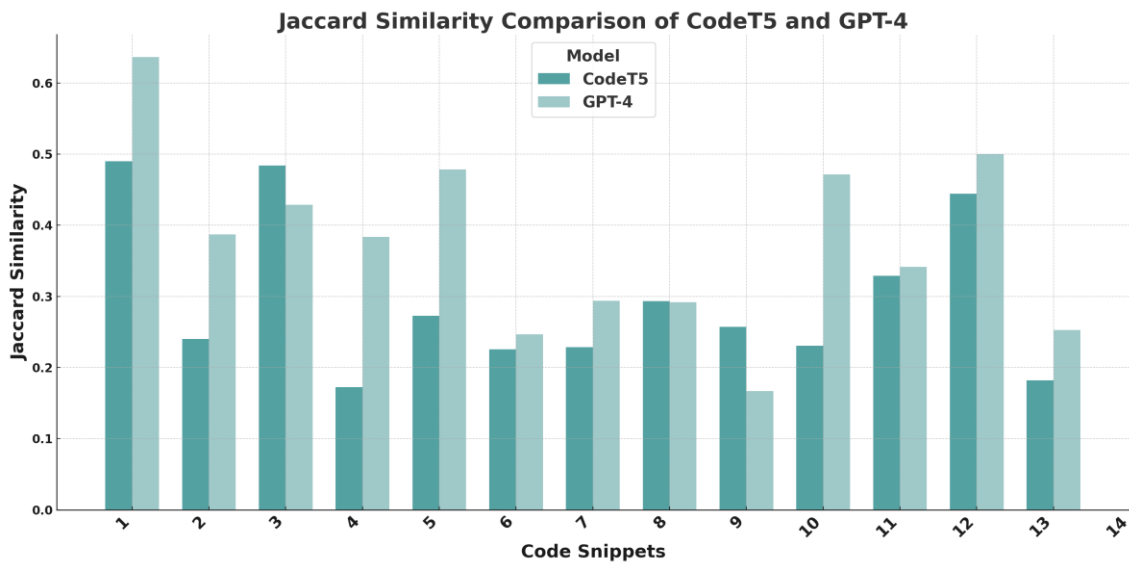


Figure 6. Jaccard Similarity Comparison of CodeT5 and GPT-4 across Test Snippets.

### ***Structural Correctness***

Structural correctness refers to the extent to which the syntactic and semantic relationships between elements in the generated model align with the intended architecture and logic defined by the reference model. In the context of SysML V2 model prototype generation, structural correctness ensures that the relationships, hierarchies, and dependencies between system components, states, and actions are captured to a high degree. This aspect is crucial in accelerating MBSE, guaranteeing minimum human intervention. To measure structural correctness, we utilize Graph Edit Distance (GED) (Bunke, 2000), a metric that quantifies the dissimilarity between two graphs by computing the minimum number of edit operations—such as node insertions, deletions, substitutions, and edge modifications—required to transform one graph into the other. For this purpose, both the generated SysML V2 model skeletons and the reference models are converted into directed graphs, where nodes represent components or actions, and edges capture relationships or transitions. For example, if a reference model contains a node representing a "Controller" component linked to a "Sensor" node via a dependency edge, but the generated model lacks this edge, GED would account for this discrepancy through a single edge insertion operation. Similarly, a mismatched component name or missing transition would correspond to a node substitution or deletion, respectively.

The GED results, illustrated in Figure 7, provide a quantitative measure of structural correctness by assessing the modifications required to align the generated SysML V2 model prototypes with their corresponding ground truth models. Smaller GED values indicate fewer structural differences and better



adherence to the reference structure. Conversely, higher values highlight cases where the generated model diverges away from the expected structure, requiring more edits to achieve parity. The variability in GED values reflects the complexity of individual samples and the strengths of the respective models. For example, CodeT5 exhibits a broader range of GED values, indicating that while it performs well on certain examples with low GED scores, it struggles with others, yielding higher discrepancies. In contrast, GPT-4 demonstrates a more consistent performance across examples, maintaining moderate GED values and avoiding extreme deviations. This suggests that CodeT5 might excel in generating precise structures under specific conditions but may lack the generalization consistency with limited fine-tuning resources.

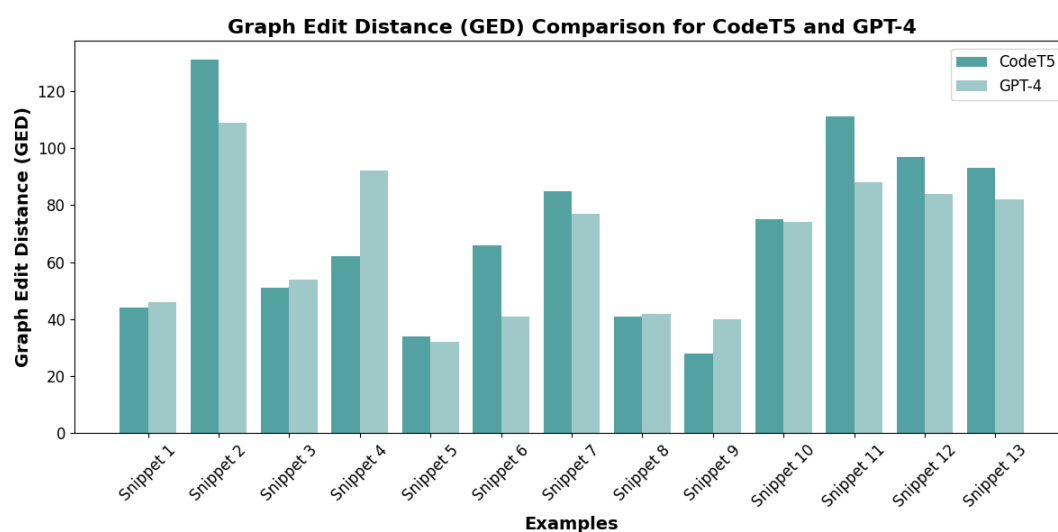


Figure 7. Graph Edit Distance (GED) Comparison between CodeT5 and GPT-4 for Structural Correctness Evaluation.

### ***Inference: Output Sample Analysis***

Table 1 shows the output of both models on two unseen natural language description inputs. The descriptions are simple and precise for better overview. The outputs reveal many differences in their approaches to SysML V2 model skeleton generation. On Test 1, GPT-4 performs better with almost a perfect model prototype while CodeT5 lacks many of the intricate details that could impact subsequent human intervention. Test 2 on the other hand, shows that CodeT5 aligns more with the natural language description and produces a skeleton that is more precise, while GPT-4 introduces a lot of noise with syntax and introduced unnecessary key words not specifically mentioned the description.



| Natural Language Description                                                                                                                                                                                                                                                                                                                               | CodeT5                                                                                                                                                                                                                                                                     | GPT-4                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Test 1:</b></p> <p>A SysML V2 model defines a Traffic Light. In the package, model a traffic light. The traffic light shall have three states: red, yellow, and green. The behavior is modeled by a state machine. The state machine is first in the state red. Then, it goes into the state yellow and then green. From green it goes into red.</p> | <p><b>Output Test 1:</b></p> <pre>package 'Traffic Light' {   import ISQ::*;   package TrafficLight {     part def Light {       state red;       then yellow;       then green;       state green;     }   } }</pre>                                                      | <p><b>Output Test 1:</b></p> <pre>package 'Traffic Light' {   part def TrafficLight {     state machine LightBehavior {       state Red;       state Yellow;       state Green;        transition RedToYellow from Red to Yellow;       transition YellowToGreen from Yellow to Green;       transition GreenToRed from Green to Red;        initial state Red;     }   } }</pre>                                                           |
| <p><b>Test 2:</b></p> <p>This SysML model defines the composition of vehicles. Define a vehicle as a part that has between two and six wheels, one or no engine, and a body. Special vehicles are electric vehicles and bicycles. Bicycles have only two wheels. Electric vehicles also have a battery.</p>                                                | <p><b>Output Test 2:</b></p> <pre>package 'Vehicle Composition' {   import ScalarValues::*;   part vehicle : Vehicle {     part wheels;     part Engine;     part body;   }   part ElectricVehicle {     part Battery;   }   part Bicycle {     define Wheels;   } }</pre> | <p><b>Output Test 2:</b></p> <pre>package 'Vehicle Composition' {   part def Vehicle {     attribute Wheels : Integer [2..6];     attribute Engine : Boolean [0..1];     part def Body;   }   part def ElectricVehicle specializes Vehicle {     part def Battery; // Electric vehicles have a battery   }   part def Bicycle specializes Vehicle {     redefine Wheels : Integer [2..2];     redefine Engine : Boolean [0..0];   } }</pre> |

Table 1. Inference: SysML V2 Model Prototypes Generated by CodeT5 and GPT-4 from Unseen Natural Language Sentences.

Here we must emphasize again on the choice of the model class. We must not simply assume that a larger model with a massive training corpus would always be the right fit. Smaller and less resource intensive models can outperform bigger models on certain use-cases. Choosing the better model depends heavily on the context and resource constraints. GPT-4's performance makes it a good candidate for general-purpose tasks where minimum human intervention is preferred, albeit with noise, even at the cost of computational resources. CodeT5, with its task-specific fine-tuning and lower resource demands, is better suited for scenarios where computational efficiency is a priority and high variability in





output quality is acceptable - emphasizing the importance of aligning the choice of an LLM class with specific project requirements. For practitioners in systems engineering, this evaluation provides a roadmap to selecting the right AI-driven tool for automating SysML V2 model prototype generation.

## Discussion

Our experiments demonstrate that different classes of LLMs like GPT-4 and CodeT5 can contribute significantly to this automation of model skeleton generation in MBSE, but their suitability depends on the task at hand. A summary of the results obtained from this study as depicted in the Table 2, provide an understanding of how two classes of advanced LLMs, GPT-4 and CodeT5, compare and perform.

### *Performance Across Various Metrics*

To interpret the evaluation metrics in Table 2, it is important to contextualize what constitutes a good similarity score or graph distance. For cosine similarity, values above 0.85 are generally considered strong indicators of semantic closeness, which GPT-4 achieves (0.85), slightly outperforming CodeT5 (0.81). Jaccard similarity, which measures token-level overlap, typically ranges between 0.4 to 0.6 for acceptable matches in structured generation tasks; both models fall at the lower end of this range (GPT-4: 0.42, CodeT5: 0.40), reflecting moderate lexical overlap.

| index | Aspect                            | GPT-4                                                        | CodeT5                                                       |
|-------|-----------------------------------|--------------------------------------------------------------|--------------------------------------------------------------|
| 0     | Model Type                        | General-purpose                                              | Task-specific                                                |
| 1     | Parameter Count (Estimated)       | Hundreds of billions                                         | 220M (base model)                                            |
| 2     | Training Dataset Size             | Diverse corpus, including natural language and code          | CodeSearchNet dataset (9 programming languages)              |
| 3     | Pre-training Domain               | General text and code                                        | Programming code                                             |
| 4     | Fine-tuning Capability            | No fine-tuning needed                                        | Requires fine-tuning                                         |
| 5     | Training Time (SysML Task)        | N/A (uses in-context learning)                               | Approx. 2 hours on NVIDIA A100                               |
| 6     | Evaluation Metrics                | Semantic similarity, syntax accuracy, structural correctness | Semantic similarity, syntax accuracy, structural correctness |
| 7     | Average Levenshtein Distance      | 1034.2                                                       | 1151.5                                                       |
| 8     | Average Cosine Similarity         | 0.85                                                         | 0.81                                                         |
| 9     | Average Jaccard Similarity        | 0.42                                                         | 0.40                                                         |
| 10    | Average Graph Edit Distance (GED) | 68.3                                                         | 74.7                                                         |

Table 2. Details and Comparisons of both LLM models.

As for Graph Edit Distance (GED), values below 50 indicate high structural similarity, while scores up to 80 might still be acceptable in complex domains like systems modeling. GPT-4's GED of 68.3 and



CodeT5's 74.7 suggest that both models produce structurally comparable outputs to the reference models, with GPT-4 showing slightly better preservation of model hierarchy.

## T-Tests

A paired t-test (also called a dependent t-test (Student, 1908) is a statistical test used to determine whether there is a significant difference between the means of two related groups. The inclusion of paired t-tests provides a statistical rigor to the analysis, enabling a deeper understanding of the observed performance differences. By testing the null hypothesis that there is no mean difference between the models' performances, the t-tests as depicted in Table 3, helps us validate whether the differences across metrics were significant.

| index | Metric                    | P-Value (p) | Interpretation                                                   | Key Insights                                                                                             |
|-------|---------------------------|-------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 0     | Levenshtein Distance      | 0.045       | Statistically significant differences in syntax accuracy.        | CodeT5 demonstrates superior performance in generating structured syntax.                                |
| 1     | Cosine Similarity         | 0.019       | Statistically significant differences in semantic alignment.     | GPT-4 excels in tasks requiring contextual understanding of natural language inputs.                     |
| 2     | Jaccard Similarity        | 0.032       | Statistically significant differences in semantic alignment.     | GPT-4 further confirms its strength in maintaining semantic relationships.                               |
| 3     | Graph Edit Distance (GED) | 0.081       | Differences are not statistically significant at the 0.05 level. | Smaller models like CodeT5 can be used when structural correctness is prioritized over semantic nuances. |

Table 3. Paired T-Test Results and Key Insights.

For example, the syntactic differences (as measured by Levenshtein Distance) were more pronounced and consistent, while the semantic alignment differences (Cosine and Jaccard Similarities) were less conclusive, reflecting variability in the tasks. The chosen significance level  $\alpha = 0.05$  serves as a threshold to decide whether the null hypothesis can be rejected. A p-value less than 0.05 indicates that the observed differences are statistically significant, while a p-value greater than or equal to 0.05 suggests that the differences may have arisen due to random chance. The t-tests confirm that larger and smaller LLMs exhibit complementary strengths in generating SysML V2 models. Statistical significance in metrics highlights meaningful performance differences, while non-significance in others points to areas where either model class may be used, regardless of their size and complexity.

While GPT-4 generally outperformed CodeT5 across most metrics and examples, there were select cases where CodeT5 produced structurally more faithful or semantically tighter SysML V2 representations. For instance, in the 'Collision Detection System' example, CodeT5 more closely aligned with the reference model's structure and port definitions, particularly preserving elements such as named ports and flow semantics. In another example, the 'Controller' package, CodeT5 captured some port binding



relationships and interface structures that GPT-4 either oversimplified or omitted. Although GPT-4 typically generated more readable and complete descriptions, its tendency to generalize sometimes led to the omission of domain-specific constraints or interface roles. These cases illustrate that smaller, fine-tuned models like CodeT5 can offer advantages when the task is tightly scoped or when the domain-specific syntax closely resembles the training data — making them valuable for controlled engineering environments where strict adherence to model grammar is critical.

### ***Correction Effort Estimate***

To address the potential gap concerning the manual effort required to correct AI-generated models, we performed a small post-hoc effort estimation based on a representative sample (Table 4). Five models generated by GPT-4 and five models generated by CodeT5 were manually reviewed and corrections were categorized into minor (naming or small structural fixes), moderate (missing relationships or additional components), and major (restructuring key elements). Estimated correction times for each category were assigned based on observed editing effort.

Based on these estimates, the average correction effort was approximately 6 minutes per model for GPT-4 outputs and 13 minutes per model for CodeT5 outputs. By comparison, manually creating a SysML V2 skeleton model of similar complexity from scratch typically required 30–45 minutes. These results suggest that even after accounting for necessary corrections, AI-generated outputs could potentially reduce initial model development time by 60–80%. However, it is important to note that this estimate is based on very small, relatively simple models, and further study is needed to validate these findings on larger or more complex systems.

| Model           | Minor Fixes | Moderate Fixes | Major Fixes | Total Estimated Effort (minutes) |
|-----------------|-------------|----------------|-------------|----------------------------------|
| GPT-4 Output 1  | 2           | 1              | 0           | 8                                |
| GPT-4 Output 2  | 1           | 0              | 0           | 2                                |
| GPT-4 Output 3  | 3           | 1              | 0           | 10                               |
| GPT-4 Output 4  | 1           | 1              | 0           | 6                                |
| GPT-4 Output 5  | 2           | 0              | 0           | 4                                |
| CodeT5 Output 1 | 2           | 2              | 1           | 18                               |
| CodeT5 Output 2 | 3           | 1              | 0           | 10                               |
| CodeT5 Output 3 | 1           | 2              | 1           | 16                               |
| CodeT5 Output 4 | 2           | 1              | 1           | 14                               |
| CodeT5 Output 5 | 1           | 2              | 0           | 10                               |

Table 4. Estimated correction effort required for AI-generated SysML V2 model outputs.



## ***Limitations of the Study***

A potential limitation of our approach lies in the implicit reliance on the style and structure of the SysML V2 training examples, which were primarily sourced from a public repository (the official SysML V2 GitHub release), and our hand-crafted samples also shared similar style and structure. These examples, while valuable for establishing syntax and structure, inherently reflect a specific modeling style, including naming conventions, element grouping, and decomposition practices. Consequently, the models generated by the fine-tuned CodeT5 or prompted GPT-4 systems tend to mirror this default style. While this is suitable for proof-of-concept experimentation, it may pose a challenge for real-world industry adoption, where organizations often follow internal MBSE style guides that differ in notation, hierarchy, or diagram preferences. In such cases, domain adaptation would require retraining or further fine-tuning of the models on organization-specific examples to ensure conformity with internal standards.

Another key limitation to consider when applying LLM-based tools for MBSE in real-world environments involves the constraints imposed by intellectual property (IP), data confidentiality, and export control regulations. Current public LLM platforms, including GPT-4, operate under usage policies that may allow prompt data to be retained or used for future model improvement, which poses a significant concern for organizations dealing with proprietary system models or regulated technologies. As a result, many industrial and academic institutions may be unable to leverage cloud-based LLMs for MBSE tasks involving sensitive content. These constraints necessitate the use of locally hosted or fine-tuned LLM instances, which, while privacy-preserving, often lack the scale, generalization capability, or up-to-date knowledge of commercial counterparts. This trade-off may impact both the quality of generated models and the adaptability of the LLM to unfamiliar modeling styles or domains. Future research and tool development should consider privacy-first LLM deployments tailored for secure MBSE workflows, especially in regulated sectors.

## **Conclusion**

This study presents a comprehensive evaluation of two distinct classes of LLMs for generating SysML V2 model skeletons from natural language descriptions. By comparing the performances of CodeT5, a smaller pre-trained transformer model fine-tuned on a small dataset, and GPT-4, a larger, general-purpose LLM, we have highlighted their complementary strengths and weaknesses across syntax accuracy, semantic alignment, and structural correctness metrics. The findings reveal that while GPT-4 performs well at understanding semantic relationships and generating semantically aligned SysML V2 models, and CodeT5 demonstrates a strong aptitude for syntax accuracy and efficient structured syntax generation. Importantly, the results of paired t-tests confirm statistically significant differences in several metrics, highlighting the distinct capabilities of these two classes of models. One key limitation of our work is the constrained resources used to fine-tune CodeT5. With access to larger, more diverse training datasets and more advanced computational resources, the performance gap between CodeT5 and GPT-



4 would likely narrow. This highlights an opportunity for future work to explore the potential of smaller models under optimal fine-tuning conditions.

From a practical perspective, smaller models like CodeT5 offer distinct advantages in resource-constrained environments, such as on-premise deployments or edge computing scenarios where computational efficiency is critical. Their ability to be fine-tuned for domain-specific tasks makes them highly versatile for targeted applications in systems engineering. Conversely, larger models like GPT-4 are ideal for scenarios requiring extensive contextual understanding, such as handling ambiguous or highly nuanced natural language descriptions, or when rapid prototyping is needed without extensive fine-tuning. The implications for systems engineering are profound. The integration of AI-powered SysML V2 model generation tools can significantly streamline workflows, reduce human effort in manual prototyping, and reduce development time. This research not only highlights the practical utility of both small and large LLMs but also emphasizes the importance of tailoring model selection to the specific needs and constraints of systems engineering projects. By leveraging the unique strengths of different classes of models, systems engineering can benefit from enhanced productivity, better resource allocation, and improved collaboration across multidisciplinary teams. We must also keep in mind that sole reliance on LLMs to produce quality models is not advised as discussed in the paper earlier. It must be best used as an accelerator in the early development phase.

## Acknowledgment

This work was funded by the BMBF Projects, KI4Boardnet and GENIAL.

## References

- Akundi, Aditya; Euresi, Daniel; Luna, Sergio; Ankobiah, Wilma; Lopes, Amit; Edinbarough, Immanuel. (2022). State of Industry 5.0—Analysis and Identification of Current Research Trends. *Applied System Innovation*, 27.
- Bunke, H. (2000). Graph Matching: Theoretical Foundations, Algorithms, and Applications. *Proceedings of Vision Interface*, 82–88.
- Chami, Mohammad; Zoghbi, Christophe; Bruel, Jean-Michel. (2019). A first step towards AI for MBSE: Generating a part of SysML models from text using AI. In *INCOSE Artificial Intelligence for Systems Engineering: 2019 Conference Proceedings*. Independently published.
- Husain, H., Codio, C., Li, D., Napoles, C., Yang, S., Yasunaga, M., & Riedel, S. (2020). *CodeSearchNet challenge: Evaluating the state of semantic code search*. Retrieved from <https://arxiv.org/abs/1909.09436>



**35<sup>th</sup>** Annual **INCOSE**  
international symposium  
hybrid event  
Ottawa, Canada  
July 26 - 31, 2025

- Jaccard, P. (1901). Étude Comparative de la Distribution Florale dans une Portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579.
- Johns, Brian; Carroll, Kristina; Medina, Casey; Lewark, Rae; Walliser, James. (2024). AI Systems Modeling Enhancer (AI-SME): Initial Investigations into a ChatGPT-enabled MBSE Modeling Assistant. *INCOSE International Symposium*, 34(1), 1149–1168.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- Manning, Christopher D.; Raghavan, Prabhakar; Schütze, Hinrich. (2008). *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Moberley, D., Rosenberg, D., & Weilkeins, T. (2024). *AI assisted MBSE with SysML. MBSE4U*. Object Management Group. (2024). *Kernel Modeling Language KerML*. Object Management Group. Retrieved from <https://www.omg.org/spec/KerML>
- OpenAI. (2023). *GPT-4 Technical Report*. OpenAI. Retrieved December 2024, from <https://openai.com/research/gpt-4>
- Riesener, M.; Dölle, C.; Becker, A.; Gorbacheva, S.; Rebentisch, E.; Schuh, G. (2021). Application of natural language processing for systematic requirement management in model-based systems engineering. *INCOSE International Symposium*, 806–815.
- Student, F. (1908). The probable error of a mean. *Biometrika*, 1-25.
- Wang, Y., Liu, H., Jiang, Z., Neubig, G., & Rajani, N. (2021). *CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation*. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- Wolf, T. D. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, (pp. 38–45). doi:<https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- Zevio, Stella. (2024). *Natural Language Processing for Requirements*. Paris: Cénotélie.



## Biography



**Khushnood Adil Rafique.** AI & MLOps Engineer and Researcher with an expected PhD in the application of machine learning and MLOps from TU Kaiserslautern (RPTU). He has a masters in Computer Science specializing in AI and a second masters in Computer Application.



**Sanan Shah.** Software developer and researcher with background in full-stack web development and systems engineering. He holds a master's degree in Computer Science from TU Kaiserslautern (RPTU). He is passionate about emerging technologies and clean code, He enjoys solving real-world problems through innovative software solutions.



**Šandor Dalecke.** Ph.D. student in computer science at the Cyber-Physical-Systems research group of Prof. Dr. Christoph Grimm at RPTU Kaiserslautern-Landau, Germany. Working on Systems Engineering with a focus on domain-expert integration and accessibility. Member of the German INCOSE society (Gesellschaft für Informatik). Special interest in education and human motivation and scientific communication.



**Christoph Grimm.** Christoph works as professor at the RPTU University Kaiserslautern-Landau. From 2006-2012 he was professor at Vienna University of Technology, 2012 he followed a call to the TU Kaiserslautern (now: RPTU for Technical University of Rhineland-Palatine). His research deals with specification and verification of discrete/continuous systems. He contributed significantly to the standardization of modeling languages.